Ownership Types

Alexandru Bolfa - 5495741

June 19, 2026

Introduction

This report showcases the **implemented features** of the ownership language and explains how the project meets the **one-person AFP requirements**. It covers the syntax, the implemented feature phases, example programs, and the required functional-programming techniques used in the implementation: **persistent data structures**, **Accessor-based lenses/traversals**, and **monad transformers/effects**.

The project README[1] is the practical guide for building and running the project. This report is the *design and architecture overview:* what was built, how the type checker and interpreter are organized, where the required techniques appear in the code, and why the **persistent borrow graph** counts as advanced usage.

Section 1 outlines the *syntax*, *implemented features*, and *pass pipeline*. Section 2 covers the **required techniques**. Section 3 describes the persistent borrow graph used for **advanced usage**. Section 4 summarizes the type and ownership checks behind the implemented features.

1 Design and Scope

Syntax

The language is a *small Rust-like subset*. It has ownership, borrowing, mutation, lists, functions, and cooperative concurrency:

```
R ::= int | bool | unit | [R]
T ::= R | &R | &mut R

S ::= let x = E | let mut x = E | P = E
    | fn f(params) -> T { S* } | { S* }
    | if E then { S* } | if E then { S* } else { S* }
    | while E { S* } | return E | E

E ::= n | True | False | x | E op E | if E then E else E
    | f(E*) | [E*] | E[E] | &P | &mut P | *E
    | spawn(f(E*)) | yield()

P ::= x | P[E] | *P
```

`unit` is the type for computations that do *not* produce useful data. It has one runtime value, written as `()`, and is used by effect-only operations such as `yield()`, `spawn(f(args))`, and list mutation.

[1] The project README is also included as Appendix B.

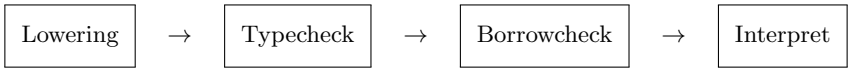
Implemented features:

Phase 0	Mutable variables, blocks and control flow, and more complex functions.
Phase 1	Affine types.
2A	Lists.
2B	Copyable primitives.
3A	Immutable borrows.
3B	Mutable borrows.
4B	Multi-threading through `spawn` and `yield`.

These features cover the **one-person phase targets**: phase 0 and phase 1, two phase-2 features (`2A`, `2B`), two phase-3 features (`3A`, `3B`), and one phase-4 feature (`4B`). Together with the **advanced persistent borrow graph** in Section 3, this covers the feature and technique requirements listed for a one-person submission.

Pipeline

The implementation is organized as a **small pass pipeline**. Expressions and statements are kept separate in Core, and assignments/borrows use a shared *place* representation such as `x`, `xs[0]`, or `*r`.



Lowering gives names unique identities, **typechecking** assigns ordinary types, **borrow checking** enforces moves and active borrows, and **interpretation** runs only after both static checks pass. Runtime references are symbolic paths into the environment, so writing through `&mut xs[0]` rebuilds the path back to `xs[0]`.

Tools and Libraries

The corresponding `.mli` files document the higher-level functions and state types exposed by each pass. The tests include **positive examples** that should run and **negative examples** that should be rejected by name resolution, typechecking, borrow checking, or runtime checks.

The implementation deliverables are the **typechecker**, **interpreter**, **positive and negative test suite**, **module documentation**, and this design overview.

The main libraries are `Base` for data structures [4], `Async` for `spawn`/`yield` [3], `Accessor` for generated record accessors [2], `monadic` for monad transformers [1], and `dune`/`ppx_jane` for building and PPX support [6, 5].

2 Implementation Techniques

For a one-person submission, the required techniques are **persistent functional data structures**, **Accessor-based lenses**, and **monad transformers**.

2.1 Persistent Functional Data Structures

Environments are *immutable* `Map.M(String)` values wrapped by `Common.Value.Env`. Updating a variable or function binding returns a new environment. Lists are also **persistent**: `src/common/plist.ml` rebuilds

only the path that changes, which matters for writes through places such as `&mut xs[0]`.

In Haskell terms, the environments are used like persistent `Map` values: an update does not mutate the old environment, but returns a new version that shares unchanged structure with the previous one. The OCaml code follows the same model with `Base.Map`. For example, binding a variable uses `Env.bind`, which returns a new map-backed environment instead of editing the old one in place.

The custom persistent list and borrow graph use the same idea for project-specific data. A list update rebuilds only the affected path, and the borrow graph operations such as `add_edge`, `remove_owner`, and `merge` all return new graph values. This is the same style as writing small immutable tree or graph transformations in Haskell.

2.2 Lenses / Traversals

Pass states are records with **generated accessors**. The local monadic wrappers expose `access` and `map_access`, so a rule updates only the field it needs:

```
M.map_access owns ~f:(Env.bind x Moved)
```

The OCaml mechanism is `ppx_accessor` plus Jane Street's `Accessor` library. Each pass state is a record marked with `[@@deriving accessors]`, which generates values such as `owns`, `graph`, `vars`, and `funs`. These generated accessors play the same role as Haskell lenses for record fields.

The helper `M.access` corresponds to a Haskell-style `view`, and `M.map_access` corresponds to applying an `over`-style update inside the current state:

```
-- Haskell shape
view owns state
modify (over owns (Env.bind x Moved))
```

The OCaml code spells this through modules and functions instead of Haskell type classes:

```
let access accessor =
  AppM.create (fun s -> Except.pure (Accessor.get accessor s, s))

let map_access accessor ~f =
  AppM.create (fun s -> Except.pure ((), Accessor.map accessor s ~f))
```

This is used throughout the passes, not just once. For example, the borrow checker updates `owns` and `graph` independently, while the typechecker updates `vars`, `muts`, `funs`, and `ret` through the same accessor pattern.

2.3 Monad Transformers / Effects

The static passes use the same structure as Haskell's `StateT AppState (Either String)`, built with `monadic`:

$$\alpha\ AppM.t = AppState.t \to ((\alpha \times AppState.t),\ string)\ result.$$

The main difference is syntactic. Haskell can write a transformer stack directly as a type alias, while the OCaml library builds the same stack through modules and functors. The Haskell shape is:

```
type AppM a = StateT AppState (Either String) a
```

The OCaml version first wraps the state type in a module, then creates a `Result` monad whose error type is `string`, and finally places state on top of that result layer:

```
module AppState = struct
  type t = env
end

module Except = Monadic.Result.Make (struct
  type t = string
end)

module AppM = Monadic.State.MakeT (Except) (AppState)
```

This gives the pass `AppM.get` and `AppM.put`, corresponding to Haskell's `get` and `put` from the state layer. It also gives `AppM.elevate`, which is the local equivalent of Haskell's `lift`. For example, throwing an error from the result layer is:

```
let fail msg = AppM.elevate (Except.error msg)
```

which corresponds to the Haskell expression:

```
fail msg = lift (Left msg)
```

Each static pass then wraps this in a small local `M` module so the rest of the pass can use `let%bind`, `get`, `put`, `fail`, `access`, and `map_access` without repeating the transformer plumbing.

The interpreter puts `Async.Deferred.t` under the same stack so `spawn` and `yield()` can suspend cooperatively. Conceptually, that interpreter stack is the OCaml equivalent of putting an asynchronous base monad under `ExceptT String` and then putting `StateT Env` on top:

```
type EvalM a = StateT Env (ExceptT String Deferred) a
```

In the OCaml code, `Result.MakeT` builds the result transformer over `Deferred`, and `State.MakeT` adds the state layer above it. The helper `lift_deferred` lifts scheduler actions such as `yield()` from `Deferred.t` into the interpreter monad.

3 Advanced Usage

For **advanced usage**, the borrow checker uses a **persistent borrow graph** implemented in `src/common/borrow_graph.ml`. The graph is carried in the state of `src/passes/borrowcheck.ml`, so each borrow-checking step receives one version of the graph and returns the next version.

The graph stores borrow relationships as *immutable edges*:

```
type owner =
  | Stored of AbsLang.ident
  | Temporary

type edge =
  { owner : owner
  ; target : target
  }
```

A target is the borrowed root variable together with the borrow kind (`&` or `&mut`). A *stored edge* means "reference variable `r` keeps root `x` borrowed", and a *temporary edge* means "this expression borrows `x` until the current statement ends". Adding, removing, merging, or clearing edges returns a **new graph**.

The core operations are small, but they are used throughout the borrow checker:

```
let add_edge edge graph =
  if List.exists graph ~f:(same_edge edge)
  then graph
  else edge :: graph

let add_owned ~owner target graph =
  add_edge { owner = Stored owner; target } graph

let merge a b =
  List.fold b ~init:a ~f:(fun graph edge -> add_edge edge graph)
```

The graph is part of the **borrow-checker state**:

```
type env =
  { owns : ownership Env.t
  ; graph : Borrow_graph.t
  }

let map_graph f = M.map_access graph ~f
```

Borrow-graph updates go through this state field. The checker calls `Borrow_graph.active` before reads, moves, assignments, and borrows to find active edges for a root. It calls `add_owned_all` when a reference variable is bound, `remove_owner` when a reference goes out of scope or is overwritten, and `add_temporary` when an expression creates a short-lived borrow. Control flow also uses the same structure: blocks call `keep_outer`, `if` expressions and statements call `merge` or `merge_stored`, and each statement finishes with `clear_temporaries`. The important property is that these operations *return new graphs instead of mutating the previous one.*

4 Type and Ownership Checks

This section summarizes the checks that support the implemented features.

4.1 Typechecking

Lowering resolves source names to unique identifiers before typechecking. The typechecker then records variable types, binding mutability, and function signatures. It checks the ordinary shape of expressions: literals, lists, indexing, conditionals, function calls, assignments, borrows, and dereferences.

Assignments are checked against the type of the target *place*. The place must be writable, so assigning to an immutable binding is rejected before the borrow checker runs. Borrowing is also typed here: `&p` produces a *shared reference*, `&mut p` requires a writable place and produces a *mutable reference*, and dereferencing requires either kind of reference.

4.2 Ownership and Borrows

Borrow checking runs *after* typechecking. It tracks whether each owned value is still available or has been moved, and it stores active borrow edges in the persistent graph described in Section 3. The copyable values are `int`, `bool`, `unit`, and shared references. Lists and mutable references are **affine**: moving one transfers ownership.

Reads, moves, assignments, and borrows all consult the active borrow graph. A move, assignment, or mutable borrow requires the root variable to have no active borrows. A shared borrow is allowed unless

there is already an active mutable borrow. Temporary borrows are cleared at the end of the statement, while stored borrows stay attached to the reference variable that owns them.

4.3 Spawning and Runtime Checks

`spawn(f(args))` is accepted only for function calls whose argument types contain no references. `yield()` has type `unit` and is handled by the interpreter's Async-based scheduler. List bounds are checked by the interpreter during execution.

References

[1] Denommus. Monadic: A monad library for OCaml. https://github.com/Denommus/monadic, 2024.

[2] Jane Street. Accessor: Nested functional data structure accessors. https://github.com/janestreet/accessor, 2026.

[3] Jane Street. Async: Jane Street's asynchronous execution library. https://github.com/janestreet/async, 2026.

[4] Jane Street. Base: Standard library for OCaml. https://github.com/janestreet/base, 2026.

[5] Jane Street. ppx_jane: Jane Street PPX rewriters. https://github.com/janestreet/ppx_jane, 2026.

[6] OCaml Dune Project. Dune: Composable build system for OCaml. https://github.com/ocaml/dune, 2026.

Appendix

A Example Programs

These are the small programs used to show the implemented feature groups. They can be run with `make run FILE=examples/<name>.lang`.

A.1 Phase 0: functions and control flow

`examples/phase0.lang` returns 120.

```
fn fact (n : int) -> int {
  if (n == 0) then {
    return 1;
  };
  return n * fact(n - 1)
};
return fact(5);
```

A.2 Phase 1: affine ownership

`examples/phase1.lang` returns 0.

```
fn first (xs : [int]) -> int {
  return xs[0]
};

let xs = [10, 20];
let ys = xs;

first(ys);

return 0;
```

A.3 2A/2B: lists and copyable primitives

`examples/phase2.lang` returns `[4, 2, 3]`.

```
let x = 1;
let y = x;
let mut xs = [x, y];

push(xs, 3);
insert(xs, 1, 4);
remove(xs, 0);

xs[1] = xs[1] + y;

return xs
```

A.4 3A: immutable borrows

`examples/phase3a.lang` returns 42.

```
fn read (x : &int) -> int {
  return *x
};

let value = 21;
let a = &value;
let b = &value;

return read(a) + read(b);
```

A.5 3B: mutable borrows

`examples/phase3b.lang` returns 42.

```
fn set (x : &mut int) -> int {
  *x = 42;
  return 0
};

let mut value = 1;

set(&mut value);

return value;
```

A.6 4B: thread-safe spawn and yield

`examples/phase4b.lang` spawns two finite cooperative tasks.

```
fn worker (mut n : int) -> unit {
  while n > 0 {
    yield();
    n = n - 1
  };
  return yield()
};

spawn(worker(2));
spawn(worker(3));
return yield()
```

B Project README

The project README is included here for completeness. It is the practical guide for installing, testing, and running the implementation.

````
# ownership-interp

An OCaml interpreter for a small Rust-like language with ownership types.

This README is for installing and running the project. The complementary report
is at `report/main.pdf`. It explains the implementation choices and how the
project maps to the grading criteria.

## Install

The project needs OCaml 5.2 or newer. For a fresh local switch:

```sh
opam switch create . ocaml-base-compiler.5.2.0 -y
eval $(opam env)
opam pin add monadic https://github.com/Denommus/monadic.git#42
    c0a97c0667ee3daec8a31b26314b696243ee8d -y
opam install . --deps-only --with-test -y
```

## Run

```sh
make build
make test
make run                            # REPL
make run FILE=examples/phase0.lang # run a file
```

Example programs live in `examples/`.
````